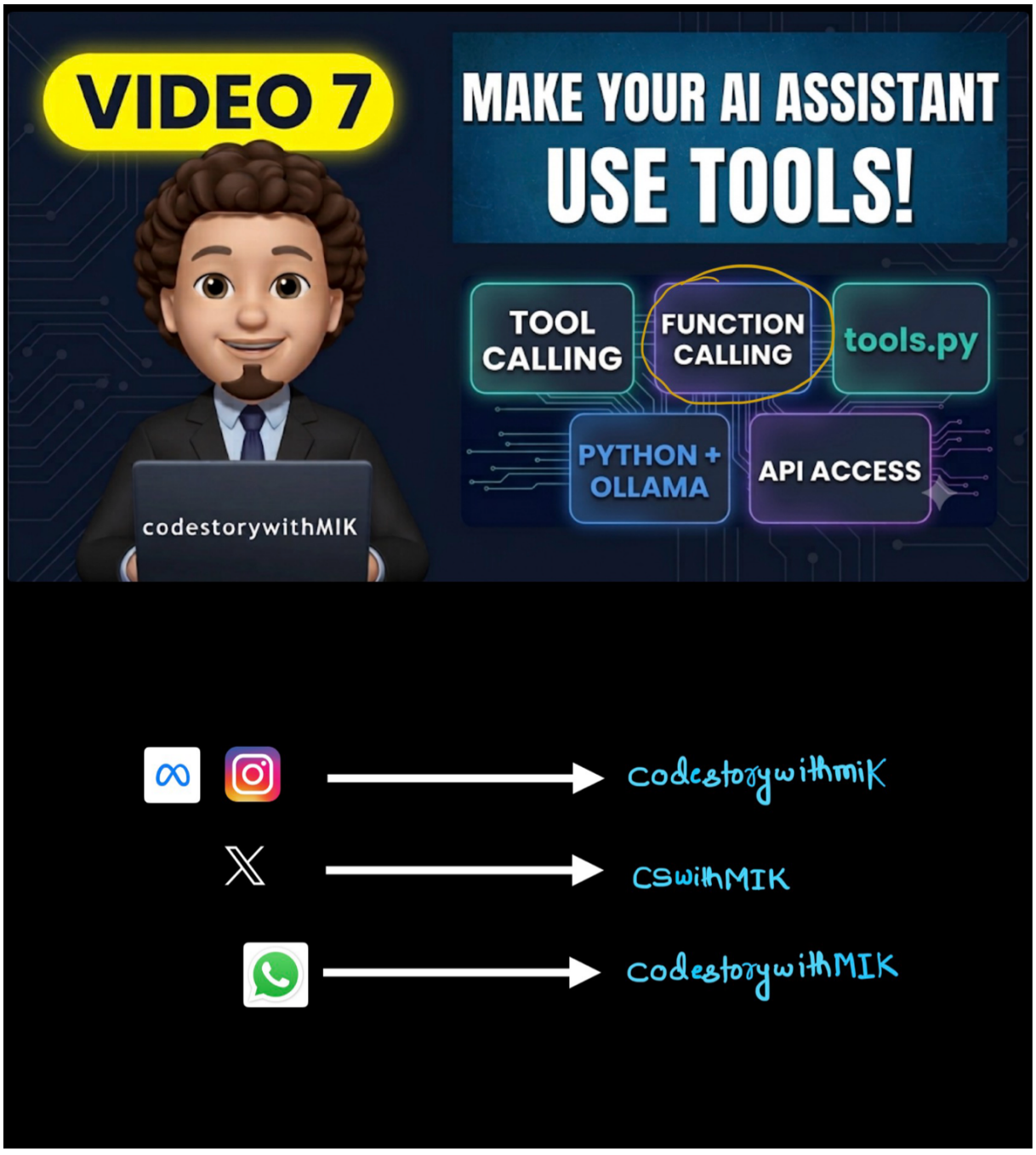
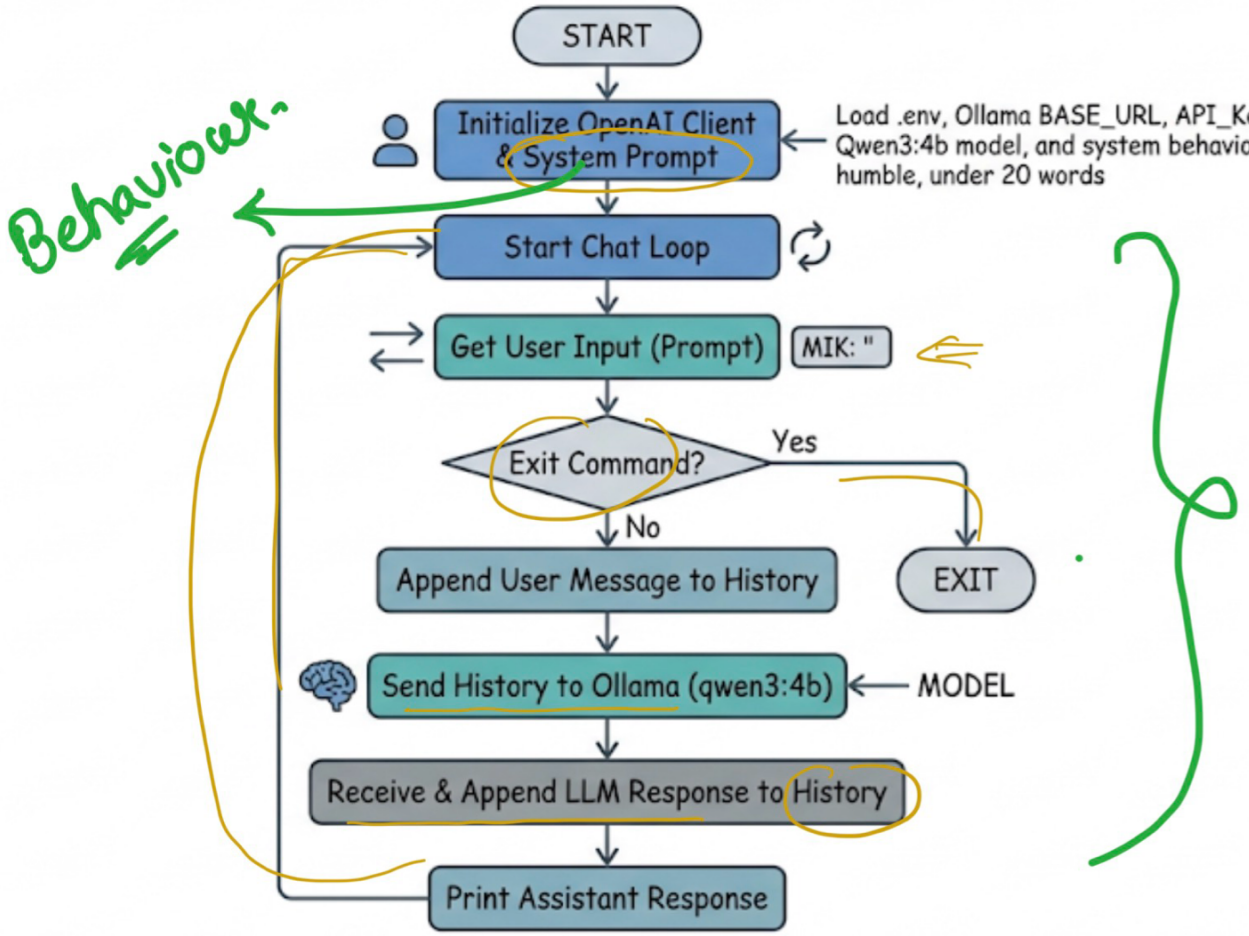

VIDEO 7
MAKE YOUR AI ASSISTANT
USE TOOLS!
codestorywithMIK
TOOL CALLING
FUNCTION CALLING
tools.py
PYTHON + OLLAMA
API ACCESS
codestorywithmiK
CSwithMIK
CodestorywithMIK

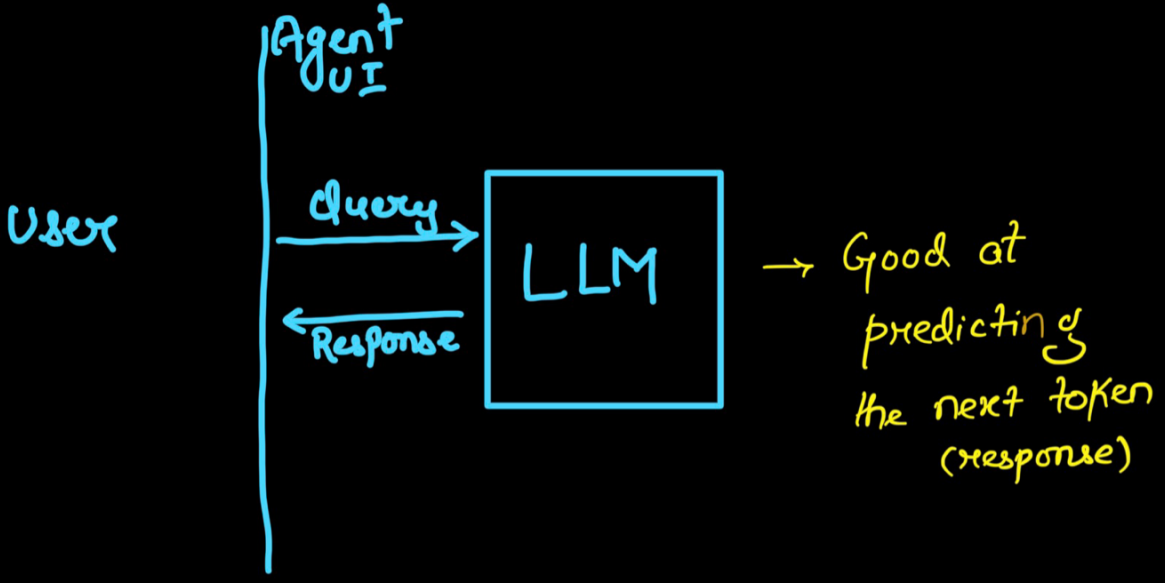
Recap : video - 6

AI Assistant Flow (Ollama & qwen3:4b)



Did you notice?

Our AI assistant cannot answer something which needs external help.



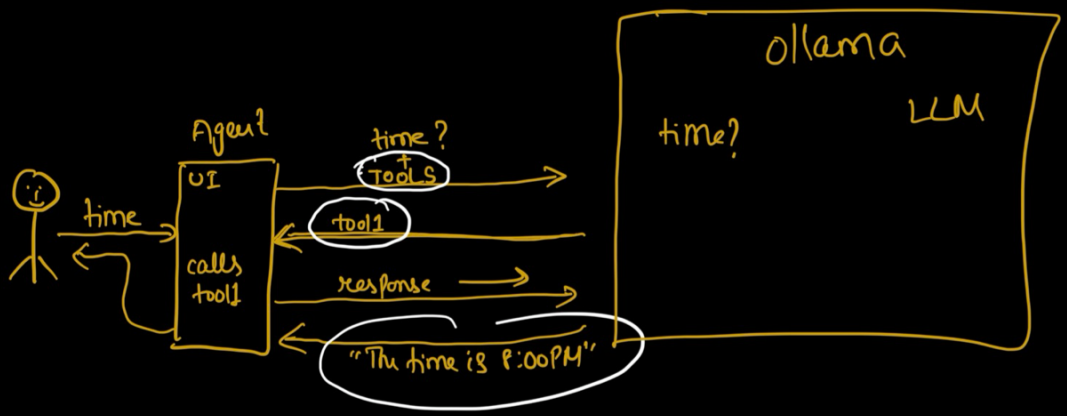
Tool Calling

(Function Calling)

(Function Calling)

It's a feature of LLM APIs where we describe our functions (tool) to the model, and the model, instead of answering, can reply with a request to call one of those functions with specific arguments.

Our code then runs the function and gives the result back to model.



Important Notes ...

1. LLM does not run the function.
   It only says which function it wants the code to run.

2. LLM gets to know about our tool (functions) because we send it along with our query

```
response = client.chat.completions.create(
    model=os.getenv("MODEL"),
    messages=messages,
    tools=TOOLS,
)
```

we will send the tools info.

OpenAI

→ Structure of one tool

```
{
    "type": "function",                 # always "function"
    "function": {
        "name": "function_name",        # name the model will call
        "description": "what it does and when to use it",
        "parameters": { ... },          # JSON Schema describing the arguments
    },
}
```

→ Format for parameter

```
"parameters": {
    "type": "object",                   # always "object" at the top level
    "properties": {                     # one entry per argument
        "argument_name_1": {
            "type": "string",
            "description": "what this argument is",
        },
        "argument_name_2": {
            "type": "string",
            "enum": ["option_a", "option_b"],      # optional: restrict to fixed choices
            "description": "what this argument is",
        },
        "argument_name_3": {
            "type": "array",
            "items": {"type": "string"},           # type of each element
            "description": "what this argument is",
        },
    },
    "required": ["argument_name_1"],    # list of mandatory arguments
}
```

"type" → "string", "array", "number", "integer", "boolean", "object"

Let's define our tools (functions) first . . .

we can have many tools, let's keep them in a separate file → tools.p

```
def get_current_time():
    return datetime.now()
```

Now most important thing :

→ Model cannot see our code.

→ we need to send details of our code to the model.

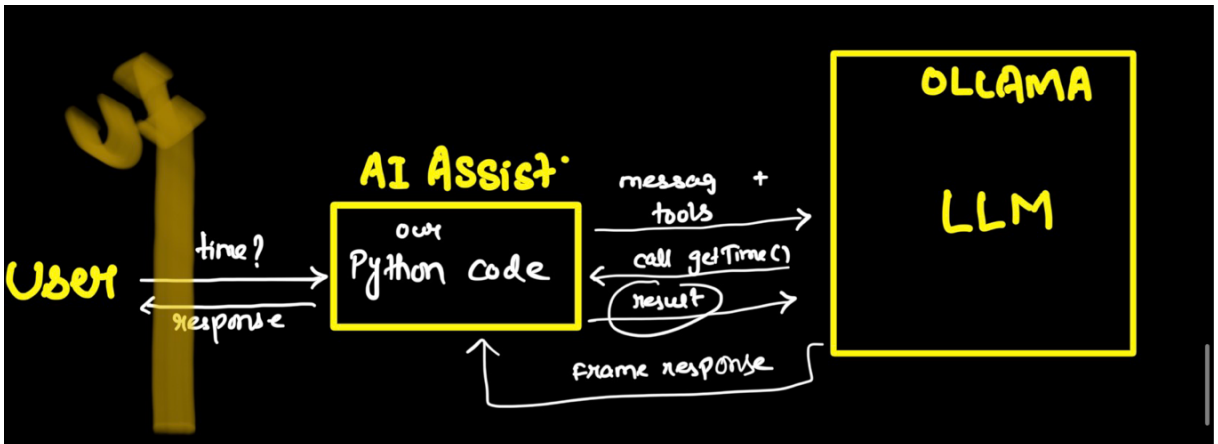
OpenAI, claude, Gemini etc. different format

TOOLS = [{ }, { }, { }]

Tool1 Details

Tool2 Details

Tool3 Details



(*) Two API calls for one user query.

→ 1st API call → To get request of tool call

→ 2nd API call → To get final response

(*) Model itself decides if tool call is needed, and which tool is needed

(This is the intelligent part LLM is helping us with)

when no tools required to answer

```
choices=[
    Choice(
        index=0,
        finish_reason='stop',
        message=ChatCompletionMessage(
            role='assistant',
            content='Hello! How can I assist you today? 😊',
            reasoning='Hmm, the user just said "Hi there"...',
            refusal=None,
            tool_calls=None,
            function_call=None,
            audio=None,
            annotations=None,
        ),
        logprobs=None,
    )
],
```

when tools required :

```
choices=[
    Choice(
        index=0,
        finish_reason='tool_calls',
        message=ChatCompletionMessage(
            role='assistant',
            content='',
            reasoning="Okay, the user is asking for the current time. Let me check the available tools. "
                      "There's a functioncalled get_current_time that returns the current date and time. "
                      "The other functions are for day of the week and listing files, which aren't relevant here. "
                      "So I should call get_current_time. No arguments needed since it defaults to the current time. "
                      "Just need to return that function in the tool_call format.\n",
            refusal=None,
            tool_calls=[
                ChatCompletionMessageFunctionToolCall(
                    id='call_uik81ss3',
                    type='function',
                    index=0,
                    function=Function(
                        name='get_current_time',
                        arguments='{}',
                    ),
                )
            ],
            function_call=None,
            audio=None,
            annotations=None,
        ),
        logprobs=None,
    )
],
```

+ [ ]

function_name = # "get_current_time"

How my code will call tools function) sent by Model ??

① First we send messages + TOOLS

```
response = client.chat.completions.create(
    model=os.getenv("MODEL"),
    messages=messages,
    tools=TOOLS,
)
```

Added this

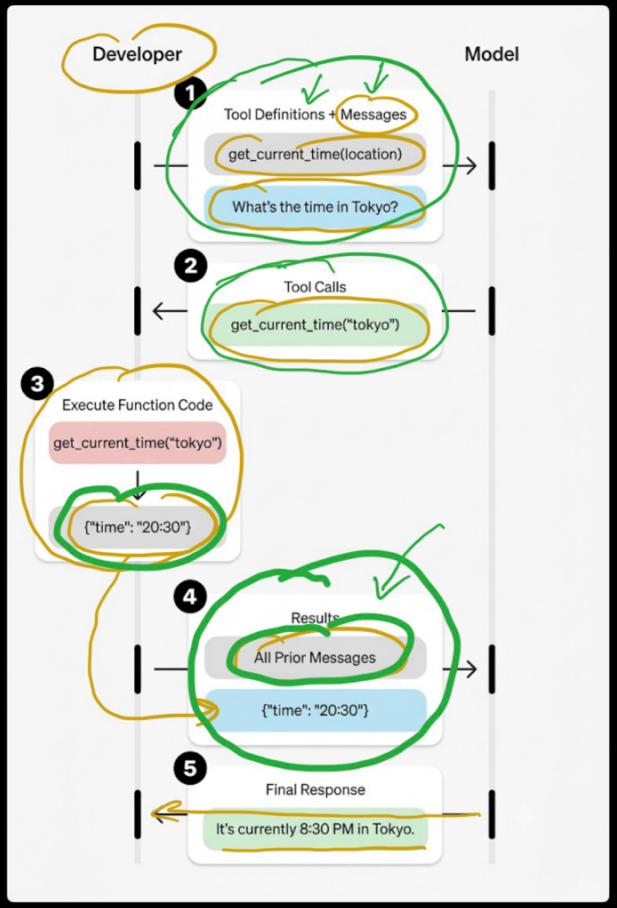
② Fetch the message from response

```
print("Assistant: " + response.choices[0].message.content) #Response from the LLM model : qwen3:4b
```

But we need too informations sent by model.

Complete Flow Diagram

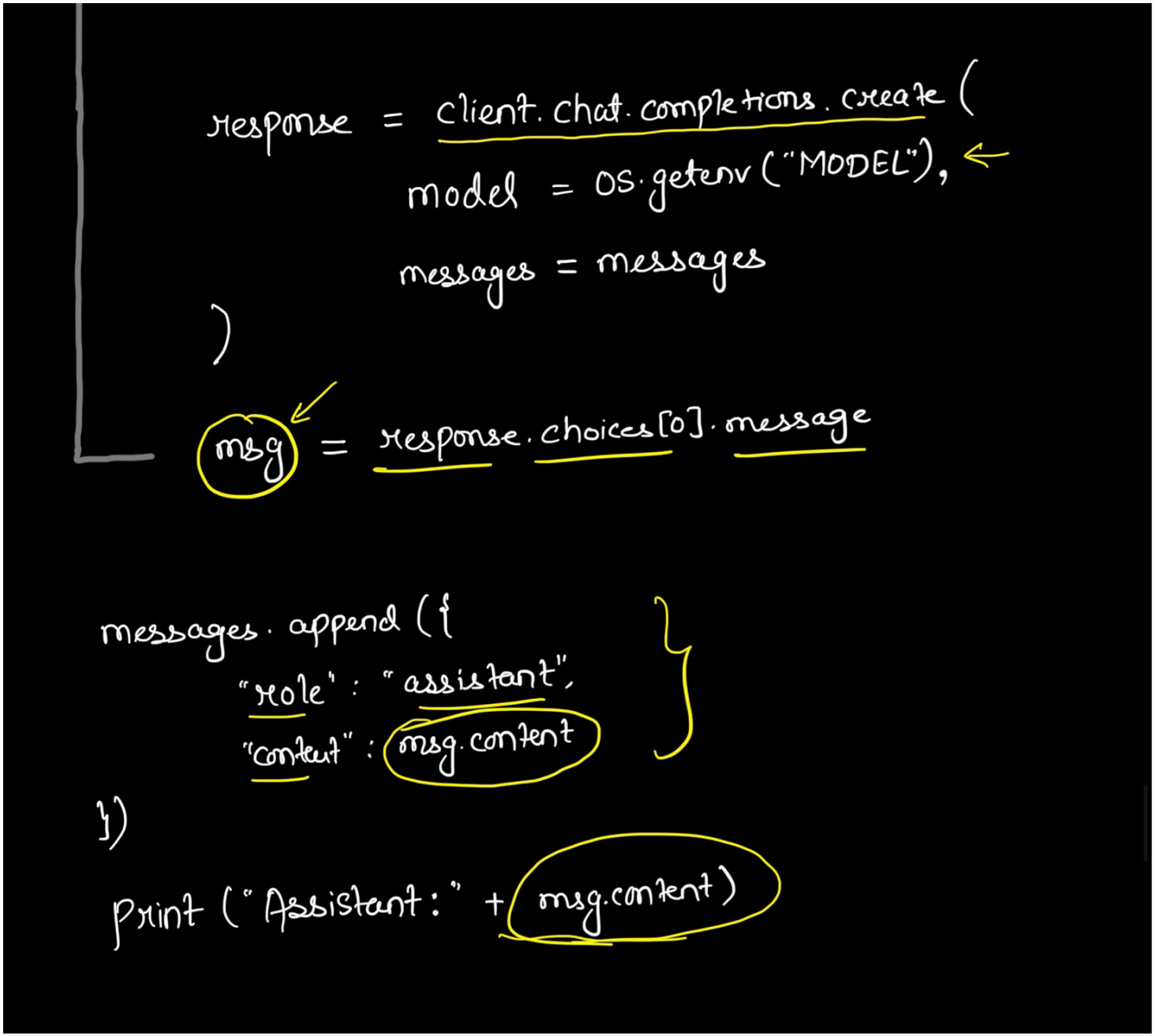
User time?

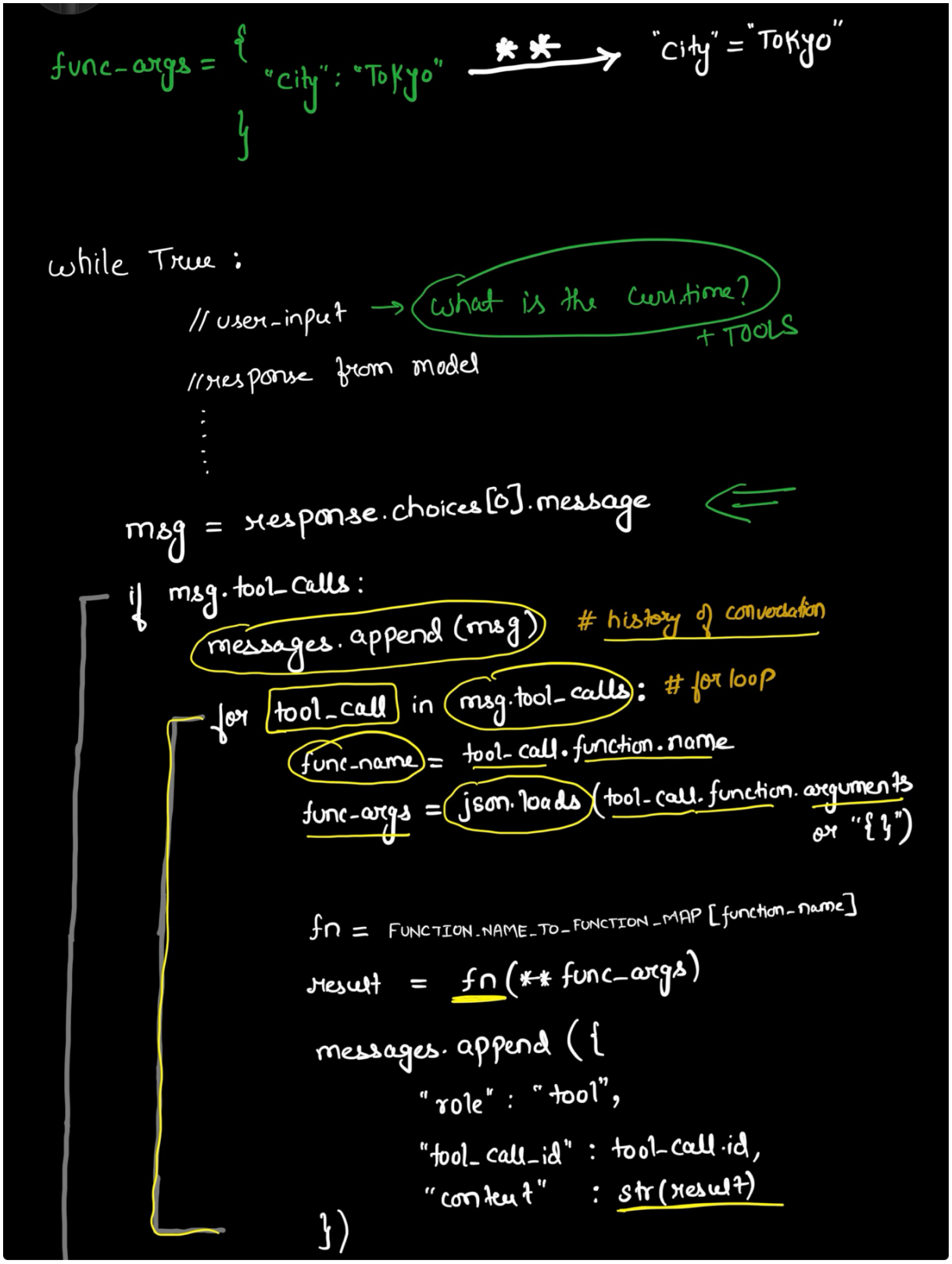


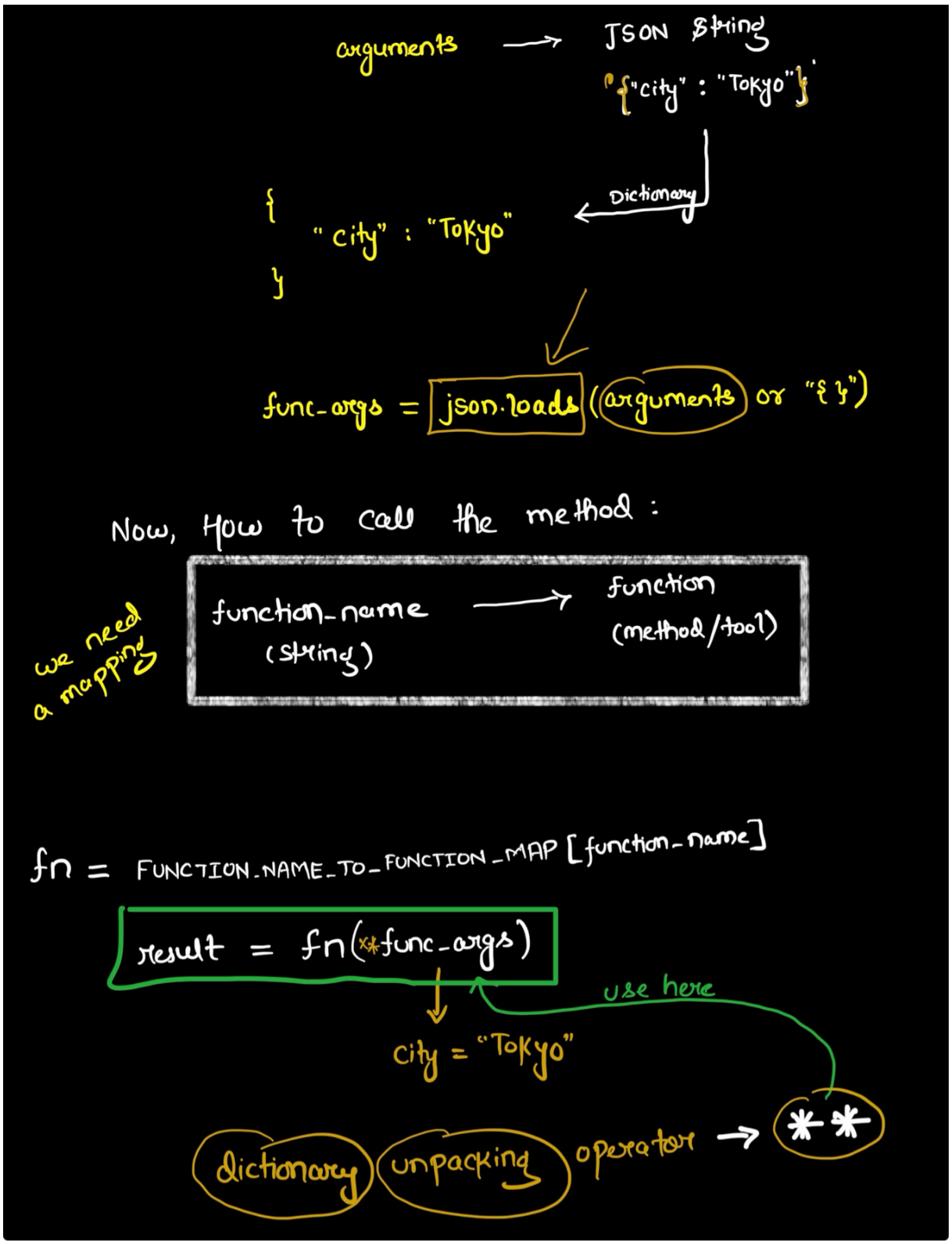
role
"user"
"assistant"
"system"
"tool"

"role": "tool",

Image Credit : openAI


response = client.chat.completions.create(
model = os.getenv("MODEL"),
messages = messages
)
msg = response.choices[0].message
messages.append({
"role": "assistant",
"content": msg.content
})
print("Assistant:" + msg.content)


func_args = { "city": "Tokyo" }
**
"city" = "Tokyo"
while True :
// user_input → what is the current time? + TOOLS
//response from model
msg = response.choices[0].message
if msg.tool_calls :
messages.append(msg) # history of conversation
for tool_call in msg.tool_calls: # for loop
func_name = tool_call.function.name
func_args = json.loads(tool_call.function.arguments or "{}")
fn = FUNCTION_NAME_TO_FUNCTION_MAP[function_name]
result = fn(** func_args)
messages.append({
"role": "tool",
"tool_call_id": tool_call.id,
"content": str(result)
})


arguments → JSON String
'{"city" : "Tokyo"}'
Dictionary
{
"city" : "Tokyo"
}
func_args = json.loads (arguments or "{}")
Now, How to call the method :
we need a mapping
function_name (string) → function (method/tool)
fn = FUNCTION_NAME_TO_FUNCTION_MAP [function_name]
result = fn(**func_args)
use here
city = "Tokyo"
dictionary unpacking operator → **